This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

GPFL: Simultaneously Learning Global and Personalized Feature Information for Personalized Federated Learning

Jianqing Zhang[1], Yang Hua[2], Hao Wang[3], Tao Song[1]
Zhengui Xue[1], Ruhui Ma[1]*, Jian Cao[1], Haibing Guan[1]
[1]Shanghai Jiao Tong University [2]Queen's University Belfast [3]Louisiana State University
{tsingz, songt333, zhenguixue, ruhuima, cao-jian, hbguan}@sjtu.edu.cn
Y.Hua@qub.ac.uk, haowang@lsu.edu

Abstract

*Federated Learning (FL) is popular for its privacy-preserving and collaborative learning capabilities. Recently, personalized FL (pFL) has received attention for its ability to address statistical heterogeneity and achieve personalization in FL. However, from the perspective of feature extraction, most existing pFL methods only focus on extracting global or personalized feature information during local training, which fails to meet the collaborative learning and personalization goals of pFL. To address this, we propose a new pFL method, named **GPFL**, to simultaneously learn global and personalized feature information on each client. We conduct extensive experiments on six datasets in three statistically heterogeneous settings and show the superiority of GPFL over ten state-of-the-art methods regarding effectiveness, scalability, fairness, stability, and privacy. Besides, GPFL mitigates overfitting and outperforms the baselines by up to 8.99% in accuracy.*

1. Introduction

To maximize the value of data generated on massive clients while protecting privacy, Federated Learning (FL), an iterative machine learning scheme, comes along with various applications [25, 21, 38, 35, 34]. Traditional FL methods focus on collaborative learning and obtaining a reasonable global model. However, in practice, one single global model cannot meet the requirements of every client and performs poorly due to statistical heterogeneity [24, 1, 50].

Recently, personalized FL (pFL) has attracted increasing attention in addressing statistical heterogeneity and achieving personalization in FL [50, 49, 63, 62]. From the view of each client, it joins FL for additional server information (*e.g.*, global model parameters) to enhance its model and address the data shortage problem. To obtain high-quality server information, each client also has to provide locally learned information for server aggregation. Thus, an ideal pFL is one kind of FL with two goals: (1) aggregating information for collaborative learning and (2) training reasonable personalized models. On the other hand, since every client is connected to the external environment and shares certain common information, the data present on each client comprises both global and personalized feature information.

However, from a feature extraction perspective, existing pFL methods only focus on one of these two goals on clients. For collaborative learning, FedRoD [8] trains the feature extractor to extract global feature information for its global objective, but it does not extract personalized feature information for personalized tasks. For personalization, FedPer [3] and FedRep [12] only use local data to train the model for the personalized objective, losing some global information during local training [10], which is not beneficial for collaborative learning. Although FedPHP [32]/ FedProto [51] utilizes global features/prototypes to guide personalized feature extraction, the quality of global features/prototypes depends on the quality of feature extractors, which is paradoxical. Poor global features/prototypes mislead feature extraction in turn.

To simultaneously learn global and personalized feature information on each client, we propose a novel pFL framework, named **GPFL**. Inspired by the category anchors that introduce extra common information in domain adaptation [65], we learn the global feature information with the guidance of global category embeddings using the *Global Category Embedding layer* (GCE). Besides, we learn personalized feature information through personalized tasks. However, learning two contrary (global *vs.* personalized) objectives is confusing, so we devise and insert the *Conditional Valve* (CoV) after the feature extractor to create a global guidance route and a personalized task route in the client model. With CoV, we learn global and personalized

*Corresponding author.

feature information separately at the same time, unlike FedRoD, FedPer, and FedRep, which only learn one kind of feature information. Besides, GPFL leverages trainable category embeddings to guide feature extraction at both the magnitude and angle levels, unlike FedPHP and FedProto, which rely on the well-trained feature extractor. Furthermore, the global category embeddings in GPFL introduce extra global information besides local data, which can mitigate the overfitting of personalized models and enhance fairness and privacy-preserving ability.

To evaluate GPFL regarding effectiveness, scalability, fairness, stability, and privacy, we compare GPFL with ten state-of-the-art (SOTA) methods on six datasets in *Computer Vision* (CV), *Natural Language Processing* (NLP), and *Internet of Things* (IoT) domains. Besides, we consider the *label skew* [40, 36, 27], *feature shift* [31], and *real world* [66, 15] settings to simulate different kinds of statistical heterogeneity in FL. Experimental results show that GPFL outperforms these baselines by up to 8.99% in accuracy. We provide the code in the supplementary materials. Overall, our key contributions are

- We emphasize the importance of achieving both collaborative learning and individualized goals in pFL and propose a pFL method GPFL that simultaneously learns the global and personalized feature information.
- We learn the global feature information through trainable category embeddings, and the additional global information in GCE mitigates the overfitting of the personalized model to local data.
- We conduct extensive experiments in the CV, NLP, and IoT domains under *label skew*, *feature shift*, and *real world* settings. The results show that our GPFL outperforms the SOTA method in terms of effectiveness, scalability, fairness, stability, and privacy.

2. Related Work & Background

2.1. Personalized Federated Learning

Meta-learning & fine-tuning. Per-FedAvg [13] and FedMeta [7] are similar methods that learn a global model with the aggregated model update trend to achieve good performance on each client with a few steps of local fine-tuning. However, the aggregated trend cannot meet the model update trends of every client.

Personalized heads. FedPer [3], FedRep [12], and FedRoD [8] split the given backbone into a feature extractor and a head. FedPer and FedRep are similar methods that only share the feature extractor between the server and clients. Different from them, each client in FedRoD owns a feature extractor and two heads. FedRoD trains the feature extractor as well as the shared head for its global objective (with the balanced softmax (BSM) loss [46]) and trains the personalized head for its personalized objective. However, it does not derive gradients, w.r.t. the feature extractor from the personalized objective, ignoring personalized feature extraction. Besides, the BSM loss is ineffective in *feature shift* settings.

Regularization. Different from FedProx [30], which regularizes the difference between local model parameters and frozen global parameters, pFedMe [49]/Ditto [29] uses a proximal term for the additional personalized models. The personalized model in Ditto benefits from the global parameter guidance. However, they ignore the global and personalized feature information extraction.

Feature extraction guidance. FedPHP aligns the features outputted by the personalized feature extractor and the global feature extractor for each sample, and FedProto aligns the feature vectors to their corresponding category prototypes. However, FedProto only guides a feature vector to be close to its corresponding prototype rather than guiding it to stay away from other prototypes, which results in the intersection of classification boundaries (see Sec. 4.2). Besides, FedProto generates prototypes based on the learned feature vectors, so the prototypes in FedProto can be uninformative without the well-trained feature extractor. This problem exacerbates when large backbones are trained from scratch in FL because they have difficulty learning good feature extractors in early iterations.

2.2. Conditional Computation

Usually, the structures of most DNNs are static during training and inference. With conditional computing techniques [6, 16, 18, 43], such as dynamic routing [18, 37, 57], a DNN can have a dynamic structure when given different conditional inputs. For example, using an auxiliary policy network, SpotTune [18] dynamically chooses which and how many blocks in a pre-trained residual network should be fine-tuned according to the input images. During inference, BlockDrop [57] executes specific layers of a residual network according to the decisions made by reinforcement learning. D^2NN [37] proposes a kind of DNN that allows selective execution through controller modules.

These methods are designed in central learning scenarios for specific tasks. Inspired by them, we propose a CoV to create global and personalized routes in the client model for global and personalized feature information extraction.

3. GPFL

3.1. Problem Statement

We have a total of N clients and client i ($i \in [N]$) generates its private data $\boldsymbol{x}_i$ (labelled by y_i) via a distinct distribution $\mathcal{D}_i$. For the personalized task, we denote the personalized objective on client i as

$$\mathcal{F}_i := \mathbb{E}_{(\boldsymbol{x}_i, y_i)\sim\mathcal{D}_i} \mathcal{L}_i(\boldsymbol{x}_i, y_i; W_i), \tag{1}$$

where $\mathcal{L}_i$ is the personalized loss function, and W_i is the parameters of all modules on client i. For all clients, our objective is

$$\{W_1, \ldots, W_N\} = \arg\min \ \mathcal{G}(\mathcal{F}_1, \ldots, \mathcal{F}_N), \qquad (2)$$

where typically $\mathcal{G}(\mathcal{F}_1, \ldots, \mathcal{F}_N) = \sum_{i\in[N]} n_i \mathcal{F}_i$, $n_i = \frac{|\mathcal{D}_i|}{\sum_{j\in[N]} |\mathcal{D}_j|}$ measures the importance of client i and $|\mathcal{D}_i|$ is the number of training samples on client i. With Eq. (1), Eq. (2) considers collaborative learning and personalization.

3.2. Method

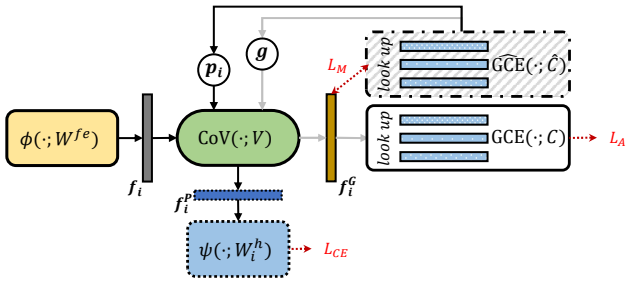


Figure 1. Illustration of client modules and data flow between them. Client i shares W^{fe}, V, C, and $\hat{C}$ while keeping W_i^h locally. Global category embeddings in $\widehat{\text{GCE}}$ are frozen before local training. We simultaneously train ϕ, CoV, GCE, and ψ in an end-to-end manner on the client. For training, we activate both the global guidance route (gray arrows) and the personalized task route (black arrows). For inference and evaluation, only the personalized task route is activated.

Overview. Focusing on the extracted features, we follow FedRep and split the backbone into a feature extractor (ϕ) and a head (ψ), where ψ is the last fully connected (FC) layers in the backbone, and ϕ represents the remaining layers. In Figure 1, to achieve the collaborative learning and personalization goals in pFL, we use CoV to transform the original feature vector $\boldsymbol{f}_i$ to two feature vectors $\boldsymbol{f}_i^G$ and $\boldsymbol{f}_i^P$ for global and personalized feature extraction, respectively. Then we learn the global feature information with the guidance of global category embeddings in GCE and learn personalized feature information through personalized tasks. Using the embedding technique [53, 64], we can get any trainable category embedding $\boldsymbol{u}$ through the *look up* operation: $\boldsymbol{u} = \text{GCE}(u; C)$, where u is a category ID. In each iteration, we share global parameters W^{fe}, V, and C among clients and obtain $\hat{C}$ by copying C after receiving C. We denote the trainable parameters as $W_i := \{W^{fe}, V, C, W_i^h\}$.

Feature extraction. We firstly obtain $\boldsymbol{f}_i$ through ϕ that maps the data samples to a lower feature space of dimension K: $\phi : \mathbb{R}^D \to \mathbb{R}^K$, where typically $D \gg K$. Formally, $\forall(\boldsymbol{x}_i, y_i) \sim \mathcal{D}_i, \boldsymbol{f}_i = \phi(\boldsymbol{x}_i; W^{fe})$.

Feature transformation. Inspired by the conditional computation techniques [18, 54, 43], we transform $\boldsymbol{f}_i$ to $\boldsymbol{f}_i^G$ and $\boldsymbol{f}_i^P$ through the affine mapping [5, 54, 43]:

$$\begin{aligned} \boldsymbol{f}_i^G &= \sigma[(\boldsymbol{\gamma} + \mathbf{1}) \odot \boldsymbol{f}_i + \boldsymbol{\beta}], \\ \boldsymbol{f}_i^P &= \sigma[(\boldsymbol{\gamma}_i + \mathbf{1}) \odot \boldsymbol{f}_i + \boldsymbol{\beta}_i], \end{aligned} \qquad (3)$$

where $\mathbf{1}$ has the same shape as $\boldsymbol{f}_i$ with all the values equal to 1 and σ is the ReLU activation function [33]. $\odot$ is the Hadamard product. $\boldsymbol{\gamma}$, $\boldsymbol{\beta}$, $\boldsymbol{\gamma}_i$, and $\boldsymbol{\beta}_i$ are generated by CoV:

$$\begin{aligned} \{\boldsymbol{\gamma}, \boldsymbol{\beta}\} &= \text{CoV}(\boldsymbol{f}_i, \boldsymbol{g}; V), \\ \{\boldsymbol{\gamma}_i, \boldsymbol{\beta}_i\} &= \text{CoV}(\boldsymbol{f}_i, \boldsymbol{p}_i; V), \end{aligned} \qquad (4)$$

where $\boldsymbol{g} \in \mathbb{R}^K$ and $\boldsymbol{p}_i \in \mathbb{R}^K$ are the personalized and global conditional input (described later), respectively. CoV consists of two sub-modules CoV_γ and CoV_β with identical structures but different parameters. Concretely, CoV_γ generates $\boldsymbol{\gamma}/\boldsymbol{\gamma}_i$ by inputting $\boldsymbol{g}/\boldsymbol{p}_i$ sequentially to an FC layer, a ReLU activation, and a layer-normalization layer [4]. CoV_β generates $\boldsymbol{\beta}/\boldsymbol{\beta}_i$ in a similar way.

Generating $\boldsymbol{g}$ and $\boldsymbol{p}_i$. $\boldsymbol{g}$ is identical among clients, while $\boldsymbol{p}_i$ contains local data distribution information. Since all the clients share the same $\hat{C}$ during local training, we can generate $\boldsymbol{g}$ by averaging all the frozen category embeddings (with dimension K):

$$\boldsymbol{g} = \frac{\sum_{u\in[U]} \widehat{\text{GCE}}(u; \hat{C})}{U}. \qquad (5)$$

For $\boldsymbol{p}_i$, we first obtain data distribution information through the statistics on client i. Specifically, we generate the proportion coefficient of category u by

$$\alpha_i^u = \mathbb{E}_{(\boldsymbol{x}_i, y_i)\sim\mathcal{D}_i} \mathbb{I}\{y_i = u\}, u \in [U], \qquad (6)$$

where $\mathbb{I}\{\cdot\}$ is the indicator function, and U is the number of all the categories among clients. Then, we have

$$\boldsymbol{p}_i = \frac{\sum_{u\in[U]} \widehat{\text{GCE}}(u; \hat{C}) \cdot \alpha_i^u}{U}. \qquad (7)$$

Angle-level global guidance. In addition to sharing model parameters between server and clients, sharing other information, such as prototype [51] and logits [61], has shown effectiveness in the literature. Thus, we propose to share the global category embeddings across clients. Inspired by the contrastive loss [28, 9, 48], we guide each feature vector to be close to its corresponding category embedding while staying away from other category embeddings, which spreads out the category embeddings during training. Formally, we have the angle-level guidance loss

$$\mathcal{L}_i^{alg} = -\log \frac{\exp\left(\text{sim}(\boldsymbol{f}_i^G, \text{GCE}(y_i; C))\right)}{\sum_{u\in[U]} \exp\left(\text{sim}(\boldsymbol{f}_i^G, \text{GCE}(u; C))\right)}, \qquad (8)$$

where $\text{sim}(\boldsymbol{f}, \boldsymbol{v}) = \frac{\boldsymbol{f}^T \boldsymbol{v}}{||\boldsymbol{f}||_2 ||\boldsymbol{v}||_2}$ is a cosine similarity function. Due to the L2-norm $||\cdot||_2$, $\text{sim}(\cdot)$ only measures the similarity between $\boldsymbol{f}_i^G$ and $\text{GCE}(u; C), u \in [U]$ at the angle level

ignoring their magnitude. Note that, all the embeddings in GCE are updated through Eq. (8).

Magnitude-level global guidance. Inspired by the proximal term that keeps the local model parameters close to the frozen global parameters [30], we keep $\boldsymbol{f}_i^G$ close to its corresponding frozen global embedding. Formally, we have the magnitude-level guidance loss

$$\mathcal{L}_i^{mlg} = ||\boldsymbol{f}_i^G, \widehat{\mathrm{GCE}}(y_i; \hat{C})||_2. \tag{9}$$

Personalized tasks. With $\boldsymbol{f}_i^P$, client i learns a head ψ that maps from the transformed feature space to the category space: $\psi : \mathbb{R}^K \to \mathbb{R}^U$. Formally, we have

$$\mathcal{L}_i^P = \ell(\psi(\boldsymbol{f}_i^P; W_i^h), y_i), \tag{10}$$

where ℓ is the *Cross Entropy* (CE) loss function [68].

Local loss function $\mathcal{L}_i$. Combining the loss mentioned above together, we have

$$\mathcal{L}_i = \mathcal{L}_i^P + \mathcal{L}_i^{alg} + \lambda \mathcal{L}_i^{mlg} + \mu||V||_2 + \mu||C||_2, \tag{11}$$

where λ and μ are hyperparameters. The entire learning process is shown in Algorithm 1.

3.3. Privacy Analysis

Following FedCG [56], we consider a semi-honest scenario where the server follows the FL protocol but may recover original data from a victim client i with its model updates through *Deep Leakage from Gradients* (DLG) [69]. Given model (or feature extractor) Φ and model updates Δ, DLG learns dummy input $\tilde{\boldsymbol{x}}$ and dummy logits (or feature vectors) output $\tilde{\boldsymbol{z}}$ by minimizing

$$\mathcal{L}^{dlg} = ||\nabla \tilde{\ell}(\Phi(\tilde{\boldsymbol{x}}), \tilde{\boldsymbol{z}}) - \Delta||^2, \tag{12}$$

where we use *Mean Squared Error* (MSE) loss [59] for $\tilde{\ell}$. Then the server can obtain the recovered input $\tilde{\boldsymbol{x}}^*$.

In GPFL, W_i^h and $\alpha_i^u, u \in [U]$ are not shared outside the client, protecting most of the private information. Without α_i^u, the CoV module can only perform transformation for the global route, like a regular layer in a backbone. The server can treat the combination of ϕ and CoV as a pseudo feature extractor $\tilde{\phi} := \phi \circ \mathrm{CoV}$ and obtain $\tilde{\boldsymbol{f}}^G := \tilde{\phi}(\tilde{\boldsymbol{x}}; W_i^{fe,t}, V_i^t)$. Compared to FedPer and FedRep which only share the feature extractor, either ϕ or $\tilde{\phi}$ learns more global information. Besides, the server can utilize GCE to calculate the logit output for category u by

$$\mathrm{logit}_u = \mathrm{sim}(\tilde{\boldsymbol{f}}^G, \mathrm{GCE}(u; C_i^t)), u \in [U]. \tag{13}$$

In other words, the server can use GCE by integrating it with $\tilde{\phi}$ as a pseudo model like the uploaded client model in FedAvg and Ditto, but with more global information. According to previous work [41], the model with more global information has a better privacy-preserving ability. We show the experimental results in Sec. 4.7.

Algorithm 1 The Learning Process in GPFL

Input: N clients with their local data; initial parameters $W^{fe,0}$, V^0, C^0; η: local learning rate; λ and μ: hyperparameters; ρ: client joining ratio; T: total training iterations.

Output: Personalized model parameters $\{W_1, \ldots, W_N\}$.

Client $i, \forall i \in [N]$ initializes its ψ and obtains $W_i^{h,0}$.
for iteration $t = 0, \ldots, T$ **do**
  Server samples a client subset $\mathcal{I}^t$ based on ρ.
  Server sends $\{W^{fe,t}, V^t, C^t\}$ to $\mathcal{I}^t$.
  for Client $i \in \mathcal{I}^t$ in parallel **do**
    ▷ **local initialization**
    Initialize ϕ, CoV, GCE with $\{W^{fe,t}, V^t, C^t\}$.
    Initialize $\widehat{\mathrm{GCE}}$ with C^t.
    Generate $\boldsymbol{g}$ and $\boldsymbol{p}_i$ by Eq. (5) and Eq. (7).
    ▷ **local training**
    Update $W^{fe,t}$, V^t, C^t, $W_i^{h,t}$ simultaneously:
      $W_i^{fe,t} \leftarrow W^{fe,t} - \eta \nabla_{W^{fe,t}} \mathcal{F}_i$;
      $V_i^t \leftarrow V^t - \eta \nabla_{V^t} \mathcal{F}_i$;
      $C_i^t \leftarrow C^t - \eta \nabla_{C^t} \mathcal{F}_i$;
      $W_i^{h,t+1} \leftarrow W_i^{h,t} - \eta \nabla_{W_i^{h,t}} \mathcal{F}_i$.
    Upload $\{W_i^{fe,t}, V_i^t, C_i^t\}$ to the server.
  ▷ **Server aggregation**
  Server calculates $n^t = \sum_{i \in \mathcal{I}^t} n_i$ and obtains
    $W^{fe,t+1} = \sum_{i \in \mathcal{I}^t} \frac{n_i}{n^t} W_i^{fe,t}$;
    $V^{t+1} = \sum_{i \in \mathcal{I}^t} \frac{n_i}{n^t} V_i^t$;
    $C^{t+1} = \sum_{i \in \mathcal{I}^t} \frac{n_i}{n^t} C_i^t$.
return $\{W_1, \ldots, W_N\}$

4. Performance Comparison

We evaluate the performance of GPFL in terms of the learned features, effectiveness, scalability, fairness, stability, and privacy. Specifically, we compare GPFL with ten SOTA methods, including FedAvg [40], FedProx [30], Per-FedAvg [13], pFedMe [49], Ditto [29], FedPer[3], FedRep [12], FedRoD [8], FedPHP [32], and FedProto [51], on CV, NLP, and IoT tasks.

4.1. Setup

Datasets. For CV tasks, we use three public datasets: Fashion-MNIST (FMNIST) [58], Cifar100 [26], and Tiny-ImageNet [11]. For NLP tasks, we use two public datasets: AG News [67] and Amazon Review [14]. For the IoT task, we use a *Human Activity Recognition* (HAR) dataset [2].

Backbones. Following work [40, 39, 17], we use a 4-layer CNN on FMNIST, Cifar100, and Tiny-ImageNet. To evaluate GPFL on a backbone larger than the 4-layer CNN, we also run ResNet-18 [19] on Tiny-ImageNet. On AG News and Amazon Review, we use the fastText [23] and the 3-layer MLP [44] as the backbones, respectively. Following

previous work [60], we use a HAR-CNN on HAR to process the sensor signal. As for the local learning rate η, we set $\eta = 0.005$ for 4-layer CNN and 3-layer MLP, set $\eta = 0.1$ for ResNet-18 and fastText, and set $\eta = 0.01$ for HAR-CNN.

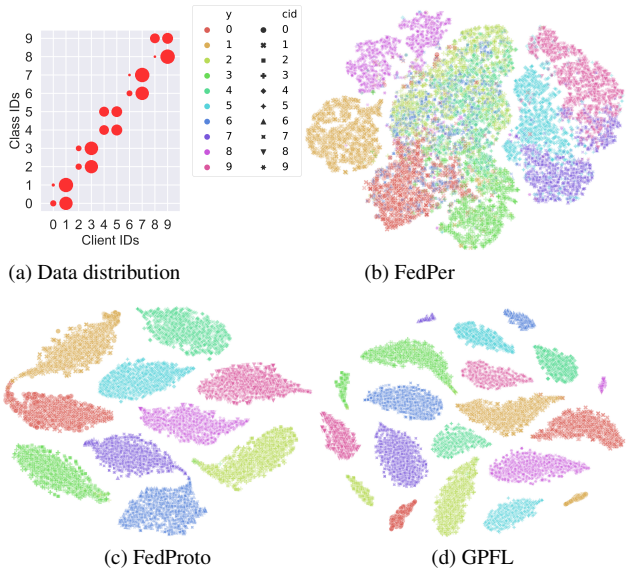
Statistically heterogeneous settings. With the above six datasets, we create three popular statistically heterogeneous settings to simulate the FL environment: *label skew* [40, 36, 27], *feature shift* [31], and *real world* [66, 15] settings. Specifically, we have two *label skew* settings: the pathological setting [40, 47] and practical setting [36, 28]. For the pathological *label skew* setting, we sample data with label amount 2/10/20 for each client on FMNIST/Cifar100/Tiny-ImageNet from a total of 10/100/200 categories, with disjoint data and different numbers of data samples. For the practical *label skew* setting, we sample data from FMNIST, Cifar100, Tiny-ImageNet, and AG News through the Dirichlet distribution [36], denoted by $Dir(\beta)$. Concretely, we sample $q_{c,i} \sim Dir(\beta)$ ($\beta = 0.1$ / $\beta = 1$ by default for CV/NLP tasks [55]) and allocate a $q_{c,i}$ proportion of the samples with label c to client i. For the *feature shift* setting, following existing methods [14, 31], we create four clients, each containing data from one domain on Amazon Review. For the *real world* setting, the sensor signal data on HAR is naturally collected and stored on 30 clients with six activities.

Implementation Details. Following pFedMe and FedRoD, unless explicitly specified, we have 20 clients with a client joining ratio $\rho = 1$. On each client, we consider 75% data as the training dataset and use the remaining 25% data for evaluation. Following pFedMe, we report the best performance of the global model for traditional FL and the best average performance across personalized models for pFL. By default, we set the batch size to 10 and the number of local epochs to 1. We run 2000 iterations with three trials for all the methods on each task and report the mean and standard deviation. For more details and experimental results (*e.g.*, the results in the *feature shift* setting with different statistical heterogeneity, *i.e.*, different β), please refer to supplementary materials.

4.2. Learned Features

Here, for easy visualization, we experiment on FMNIST in the pathological *label skew* setting using ten clients and keep other settings constant. As shown in Figure 2, we use t-SNE [52] to visualize the feature vectors extracted by FedPer, FedProto, and our GPFL.

The classification boundary is not discriminative in FedPer and the boundaries in FedProto intersect, *e.g.*, the boundaries of category 0 and 1 in Figure 2c, while Figure 2d has no intersection. By simultaneously considering global guidance and personalized tasks, GPFL can learn discriminative features and distinguish the data distribution of each client, revealing excellent personalization performance of GPFL in feature extraction.



(a) Data distribution (b) FedPer

(c) FedProto (d) GPFL

Figure 2. (a): Data distribution on each client; the size of red circles means the number of samples. (b), (c), and (d): t-SNE visualizations of feature vectors on FMNIST with 10 clients; "cid" means client ID. *Best viewed in color.*

4.3. Effectiveness

Then, we compare our GPFL with baselines under the *label skew*, *feature shift*, and *real world* settings. Due to the limited space, we use "TINY" and "TINY*" to represent using the 4-layer CNN and ResNet-18 (trained from scratch) on Tiny-ImageNet, respectively.

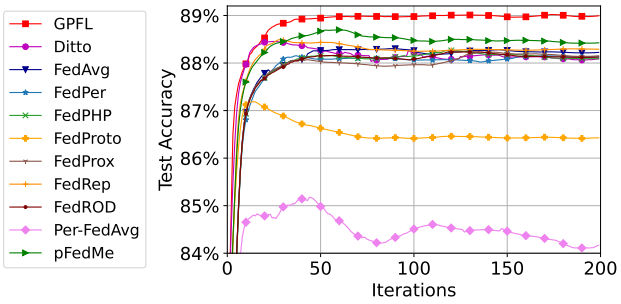
***Label skew* settings.** We show the results regarding the *label skew* settings in Table 1. GPFL achieves superior performance in both pathological and practical settings. Concretely, in the practical setting on Cifar100, GPFL outperforms the best baseline Ditto by **8.99%** with only 0.31% standard deviation. FedRep performs well on Tiny-ImageNet in the pathological setting, but GPFL outperforms it by 6.10% in the practical setting. Since there are many embeddings of tokens in NLP tasks, the feature guidance per category in GPFL and FedProto is beneficial for embedding learning in the *label skew* setting. GPFL achieves 1.63% improvement over FedProto and outperforms other baselines by 5.72% on AG News.

Next, we point out why GPFL outperforms other baselines with the experimental results. (1) **GPFL v.s. FedPer & FedRep & FedRoD**: On each client, the feature extractor trained in FedPer and FedRep only learns personalized feature information, while the feature extractor in FedRoD only learns the global one. Unlike them, GPFL locally learns both the global and personalized feature information, so GPFL outperforms FedPer/FedRep/FedRoD by 12.23%/9.47%/10.92% on Cifar100 in the practical setting. (2) **GPFL v.s. FedProto & FedPHP**: FedPHP/FedProto guides feature extraction with global features/prototypes

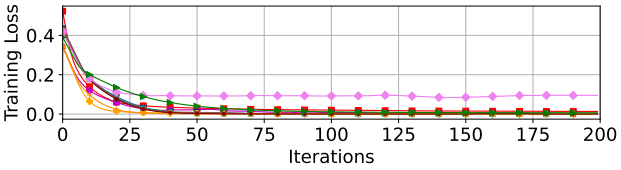
Table 1. The test accuracy (%) on the CV and NLP tasks in *label skew* settings.

Settings	Pathological *Label Skew* Setting			Practical *Label Skew* Setting				
	FMNIST	Cifar100	TINY	FMNIST	Cifar100	TINY	TINY*	AG News
FedAvg	80.41±0.08	25.98±0.13	14.20±0.47	85.85±0.19	31.89±0.47	19.46±0.20	19.45±0.13	87.12±0.19
FedProx	78.08±0.15	25.94±0.16	13.85±0.25	85.63±0.57	31.99±0.41	19.37±0.22	19.27±0.23	87.21±0.13
Per-FedAvg	99.18±0.08	56.80±0.26	28.06±0.40	95.10±0.10	44.28±0.33	25.07±0.07	21.81±0.54	87.08±0.26
pFedMe	99.35±0.14	58.20±0.14	27.71±0.40	97.25±0.17	47.34±0.46	26.93±0.19	33.44±0.33	87.08±0.18
Ditto	99.44±0.06	67.23±0.07	39.90±0.42	97.47±0.04	52.87±0.64	32.15±0.04	35.92±0.43	91.89±0.17
FedPer	99.47±0.03	63.53±0.21	39.80±0.39	97.44±0.06	49.63±0.54	33.84±0.34	38.45±0.85	91.85±0.24
FedRep	99.56±0.03	67.56±0.31	40.85±0.37	97.56±0.04	52.39±0.35	37.27±0.20	39.95±0.61	92.25±0.20
FedRoD	99.52±0.05	62.30±0.02	37.95±0.22	97.52±0.04	50.94±0.11	36.43±0.05	37.99±0.26	92.16±0.12
FedPHP	99.30±0.13	63.09±0.04	37.06±0.57	97.38±0.16	50.52±0.16	35.69±3.26	29.90±0.51	90.52±0.19
FedProto	99.49±0.04	69.18±0.03	36.78±0.07	97.40±0.02	52.70±0.33	31.21±0.16	26.38±0.40	96.34±0.58
GPFL	**99.85±0.08**	**71.78±0.26**	**44.58±0.06**	**97.81±0.09**	**61.86±0.31**	**43.37±0.53**	**43.70±0.44**	**97.97±0.14**

throughout the FL process, but the large backbone ResNet-18 cannot learn to extract features well in early iterations. Then, poor global features and prototypes mislead local training, so FedPHP and FedProto achieve low accuracy on the large backbone ResNet-18. Besides, due to the classification boundary intersection, FedProto performs worse on the dataset with more categories (*e.g.*, Tiny-ImageNet with 200 categories). By sharing the feature extractor and guiding the feature extraction with the trainable global embeddings, GPFL outperforms FedProto by 17.32% on TINY*.



(a) Test accuracy curves in the *feature shift* setting.



(b) Training loss curves in the *feature shift* setting.

Figure 3. Curves (smoothed) on Amazon Review dataset.

***Feature shift* setting.** Using the Amazon Review dataset, each client contains the data from one domain with the personalized task of classifying the samples into positive or negative emotions. In other words, the data on every client belong to two categories, which is different from the *label skew* settings. In Figure 3a, we find that the traditional FL methods FedAvg and FedProx achieve good performance with a little gap compared to the pFL methods. It means that traditional FL methods suffer more in the *label skew* setting than in the *feature shift* setting. Besides, FedAvg can maintain its performance after reaching the best accuracy. In contrast, the accuracy curves of several pFL methods, including Ditto, pFedMe, FedRep, FedProto, and Per-FedAvg, exhibit a drop in accuracy after reaching the peak accuracy, despite their training having converged (Figure 3b), which means overfitting. FedProto and Per-FedAvg achieve poor performance with a significant drop. On the contrary, GPFL performs the best and maintains its performance when converged, as the global information in GCE mitigates the overfitting of personalized models. FedRoD does not show superiority here, as its BSM loss is only designed for the *label skew* settings to tackle statistical heterogeneity.

***Real world* setting.** From Table 2, we find that the regularization-based pFL methods (pFedMe and Ditto) perform well on HAR and the regularization-based traditional FL method FedProx outperforms FedAvg by 1.14%. However, most other pFL methods, *e.g.*, Per-FedAvg, FedPer, FedRep, FedPHP, and FedProto perform worse than traditional FL methods (FedAvg and FedProx). FedAvg outperforms Per-FedAvg and FedPer by 10.08% and 11.62%, respectively. The GCE stores the global embedding of each activity, which guides the feature extractor to extract the global characteristic of one kind of human activity. Meanwhile, the personalized task can also guide the feature extractor to extract user-specific characteristics. Thus, our GPFL outperforms all baselines by up to 2.19% in this scenario.

4.4. Scalability

Based on MOON [28], we split the Cifar100 dataset into 20/30/50/100/500 sub-datasets to form 20/30/50/100/500 clients, respectively, in the practical *label skew* settings ($\beta =$

Table 2. The test accuracy (%) on the IoT task regarding effectiveness and the CV task regarding scalability.

Tasks	**Effectiveness**	**Scalability**					
Clients	$N = 30$	$N = 30$	$N = 50$	$N = 100$	$N = 500$	$N = 10\|50$	$N = 30\|50$
FedAvg	87.20±0.27	31.15±0.05	31.90±0.27	31.95±0.37	29.51±0.73	25.28±0.32	29.04±0.21
FedProx	88.34±0.24	31.21±0.08	31.94±0.30	31.97±0.24	29.84±0.81	25.65±0.34	29.04±0.36
Per-FedAvg	77.12±0.17	41.57±0.21	44.31±0.20	36.07±0.24	/	40.20±0.21	42.96±0.42
pFedMe	91.57±0.12	47.04±0.28	48.36±0.64	46.45±0.18	31.30±0.89	40.27±0.54	42.19±0.38
Ditto	91.53±0.09	52.53±0.42	54.22±0.04	52.89±0.22	30.24±0.72	48.23±0.35	50.98±0.29
FedPer	75.58±0.13	44.98±0.20	44.22±0.18	40.37±0.41	30.56±0.59	43.64±0.42	43.54±0.43
FedRep	80.44±0.42	50.24±0.01	47.41±0.18	44.61±0.20	31.92±0.71	46.85±0.12	47.63±0.26
FedRoD	89.91±0.23	50.11±0.03	49.38±0.01	46.65±0.22	34.61±0.98	46.32±0.02	49.15±0.12
FedPHP	87.94±0.54	49.28±0.06	52.44±0.16	49.70±0.31	30.26±0.84	45.71±0.21	48.65±0.24
FedProto	84.73±0.32	52.32±0.18	50.29±0.18	47.11±0.15	34.91±0.83	49.23±0.28	50.49±0.32
GPFL	**93.76±0.16**	**60.96±0.65**	**60.98±0.32**	**57.11±0.21**	**37.28±0.63**	**53.26±0.39**	**59.18±0.53**

0.1). We have shown the results for 20 clients in Table 1, so we only show the results for 30/50/100/500 clients in Table 2. The meta-learning in Per-FedAvg requires at least two batches of data, which is invalid on some clients in our unbalanced settings when $N = 500$. Since the total sample amount is constant on Cifar100, the average sample amount per client decreases when N increases and it is unreasonable to compare results between different N.

In Table 2, GPFL still outperforms other baselines with various N. When $N = 500$, it is hard to achieve personalization without enough client data for pFL methods. Many pFL baselines, *e.g.*, Ditto, FedPer, and FedPHP perform similarly with FedAvg in this situation. Since GCE provides extra information and relieves the data shortage issue on each client, GPFL outperforms all baselines.

In practice, clients join the FL process with their data, and more clients means more total data in FL. To simulate a more realistic scenario, we randomly select 10/30/50 clients from the 50 clients generated above to form a setting with a total of 10/30/50 clients, denoted by $N = 10|50$ / $N = 30|50$ / $N = 50$. In these settings, as shown in Table 2, all the methods benefit from the client amount increase, and GPFL still outperforms other baselines, showing the scalability of GPFL in practice.

4.5. Fairness

According to the work [45], personalization may result in poorer performance on some devices despite improving the average. It is essential to improve both performance and fairness when designing a pFL method. Here, following Ditto [29], we evaluate the fairness of all the methods through the standard deviation of the local accuracy on clients when reaching the best-averaged accuracy (*i.e.*, the test accuracy mentioned above), as shown in Table 3. To weaken the effect of the test accuracy magnitude for a more fair comparison, we also follow the work [22] to use the coefficient of variation metric for fairness evaluation.

Table 3. The fairness, *i.e.*, standard deviation (%, ↓) [the coefficient of variation ($\times 10^{-2}$, ↓)] of the local accuracy on clients when reaching the best test accuracy on the CV, NLP, and IoT tasks in pathological *label skew* (paLS), practical *label skew* (prLS, $\beta = 0.1$), *feature shift*, and *real world* settings.

Settings	**paLS**	**prLS**	***Feature Shift***	***Real World***
Clients	$N = 20$	$N = 100$	$N = 4$	$N = 30$
Datasets	TINY	Cifar100	Amazon Review	HAR
FedAvg	3.57 [25.14]	7.06 [22.10]	1.62 [1.84]	17.10 [19.61]
FedProx	3.51 [25.34]	7.08 [22.15]	1.60 [1.81]	17.35 [19.64]
Per-FedAvg	3.27 [11.65]	8.13 [22.54]	2.82 [3.29]	14.15 [18.35]
pFedMe	3.36 [12.12]	8.19 [17.63]	1.99 [2.25]	12.65 [13.81]
Ditto	3.84 [9.62]	9.89 [18.70]	2.12 [2.40]	13.20 [14.42]
FedPer	3.39 [8.51]	8.91 [22.07]	2.18 [2.47]	19.49 [25.79]
FedRep	3.53 [8.64]	8.99 [20.15]	2.15 [2.43]	21.16 [26.30]
FedRoD	3.46 [9.12]	8.87 [19.01]	2.24 [2.54]	16.93 [18.83]
FedPHP	3.81 [10.28]	9.45 [19.01]	1.96 [2.22]	13.81 [15.70]
FedProto	4.13 [11.23]	9.98 [21.18]	1.82 [2.08]	11.77 [13.89]
GPFL	**3.21 [7.20]**	**8.05 [14.10]**	**1.62 [1.80]**	**8.42 [8.98]**

In Table 3, our GPFL outperforms other pFL methods, especially in the *real world* setting and the settings with many clients, because sharing global information among clients promotes fairness. By learning both the global and personalized feature information, GPFL achieves a higher accuracy with lower discrimination compared to FedPer, FedRep, and FedRoD, which focus on learning only one kind of feature information during local training. The traditional FL methods achieve a low standard deviation but a high coefficient of variation, as their personalization performance is poor. Clients in FedProto only share prototypes but keep the model parameters secret, which limits the capacity of global information and leads to low fairness.

4.6. Stability

Table 4. The test accuracy (%) of the pFL methods on Cifar100 with $N = 50$, $\beta = 0.1$, and $\rho \leq 1$.

	$\rho = 1$	$\rho \in [0.5, 1]$	$\rho \in [0.1, 1]$
Per-FedAvg	44.31±0.20	43.66±1.38	43.63±1.07
pFedMe	48.36±0.64	43.28±0.85	41.71±1.02
Ditto	50.59±0.22	49.78±0.36	48.33±3.27
FedPer	44.22±0.18	44.12±0.21	44.07±0.27
FedRep	47.41±0.18	46.93±0.21	46.61±0.22
FedRoD	49.38±0.01	49.07±0.43	47.80±1.35
FedPHP	50.23±0.12	45.19±0.07	44.43±0.12
FedProto	50.29±0.18	49.45±0.21	46.05±4.03
GPFL	**60.98±0.32**	**60.60±0.51**	**60.04±0.28**

In real world scenarios, some clients cannot join the whole FL process because of low battery, lack of computation resources, unstable network, *etc*. Here, we simulate this scenario by varying the client joining ratio ρ in every iteration on the Cifar100 dataset. Specifically, we uniformly sample a value for ρ within the given range in each iteration.

In Table 4, our GPFL still maintains its superiority among the pFL methods, while some baselines perform worse with a more extensive range of ρ. For example, pFedMe and FedPHP drop 6.65% and 5.80% from $\rho = 1$ to $\rho \in [0.1, 1]$, respectively. Furthermore, Per-FedAvg, pFedMe, Ditto, FedRoD, and FedProto achieve erratic performance (large standard deviations) in dynamic settings.

4.7. Privacy

Table 5. PSNR (dB, ↓) values for privacy evaluation on Cifar100 in *label skew* setting. "Fed" is omitted for method names.

Per-*fe*	Rep-*fe*	Avg-*fe*	Avg	RoD-*fe*	RoD	GPFL-*fe*	GPFL-*sfe*	GPFL-*sm*
7.94	7.73	6.92	7.30	6.82	7.48	6.56	**6.41**	6.71

Following FedCG, we evaluate the privacy-preserving ability of GPFL with representative baselines in *Peak Signal-to-Noise Ratio* (PSNR) [20]. For FedPer, FedRep, FedAvg, and FedRoD, we conduct DLG attack for feature extractor (with suffix "-*fe*"). For FedAvg and FedRoD, we also conduct DLG attacks for the entire model. For GPFL, we conduct DLG attack for feature extractor, pseudo feature extractor (with suffix "-*sfe*"), and pseudo model (with suffix "-*sm*").

In Table 5, all the PSNR values that correspond to GPFL are smaller than other baselines, which supports the analysis in Sec. 3.3. Since the parameters of the last FC layer in a backbone also contain class information [42], the entire model is more susceptible to privacy leakage than the feature extractor in FedAvg and FedRoD. However, due to the global characteristic of GCE, GPFL-*sm* still maintains the privacy-preserving ability.

5. Ablation Study

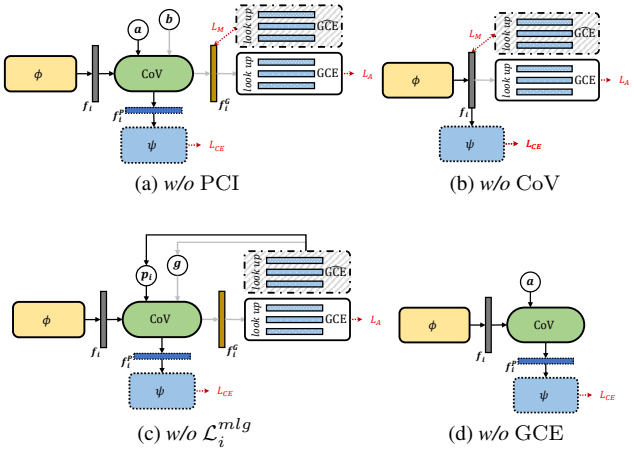


Figure 4. Illustration of variants for ablation study.

To further show the effectiveness of each proposed module, we remove them from GPFL and create five variants ("*w/o*" is short for "without"): (a) *w/o* personalized conditional input (PCI), (b) *w/o* CoV, (c) *w/o* $\mathcal{L}_i^{mlg}$, (d) *w/o* GCE ($\widehat{\text{GCE}}$ also disappears), and (e) *w/o* CoV & GCE (FedPer). As shown in Figure 4, we input the constant vector $\boldsymbol{a}$ and $\boldsymbol{b}$ instead of the dynamic $\boldsymbol{p}_i$ and $\boldsymbol{g}$ to remove PCI. We report the test accuracy of GPFL and its five variants in Table 6.

Table 6. The accuracy (%) of GPFL and its variants on TINY*.

GPFL	***w/o*** PCI	***w/o*** CoV	***w/o*** $\mathcal{L}_i^{mlg}$	***w/o*** GCE	**FedPer**
43.70	42.74	40.23	41.72	39.48	38.45

We analyze each module according to Table 6. (a) Like $\boldsymbol{g}$, $\boldsymbol{a}$ and $\boldsymbol{b}$ are identical across clients, so GPFL inputs local data distribution information for CoV while *w/o* PCI does not. With this personalized information, GPFL performs better than *w/o* PCI. Even with the identical $\boldsymbol{a}$/$\boldsymbol{b}$, *w/o* PCI still performs well thanks to end-to-end training. (b) Removing CoV causes a 3.47% accuracy decrease, as guiding one feature vector to learn both global and personalized information simultaneously is confusing. (c) The accuracy gap between GPFL and *w/o* $\mathcal{L}_i^{mlg}$ shows the effectiveness of the magnitude-level global guidance, since *w/o* $\mathcal{L}_i^{mlg}$ only removes the $\mathcal{L}_i^{mlg}$ objective. (d) The accuracy decreases further when we remove the angle-level global guidance. Without learning global information during local training, the accuracy drops by 4.22%. As the trainable affine mapping adaptively adjusts the original features, *w/o* GCE still slightly outperforms FedPer.

6. Conclusion

For the collaborative learning and personalization goals of pFL, we propose GPFL to simultaneously learn global and personalized information on the client. We show the superiority of GPFL through extensive experiments regarding effectiveness, scalability, fairness, stability, and privacy. GPFL outperforms SOTA pFL methods by a large margin. Besides, we show the effectiveness of each proposed module.

Acknowledgements

This work was supported in part by the Shanghai Key Laboratory of Scalable Computing and Systems, National Key R&D Program of China (2022YFB4402102), Internet of Things special subject program, China Institute of IoT (Wuxi), Wuxi IoT Innovation Promotion Center (2022SP-T13-C), Industry-university-research Cooperation Funding Project from the Eighth Research Institute in China Aerospace Science and Technology Corporation (Shanghai) (USCAST2022-17), and Intel Corporation (UFunding 12679). This work was also partially supported by the Program of Technology Innovation of the Science and Technology Commission of Shanghai Municipality (Granted No. 21511104700 and 22DZ1100103).

References

[1] Zareen Alamgir, Farwa K Khan, and Saira Karim. Federated Recommenders: Methods, Challenges and Future. *Cluster Computing*, 25(6):4075–4096, 2022. 1

[2] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L Reyes-Ortiz. Human activity recognition on smartphones using a multiclass hardware-friendly support vector machine. In *Ambient Assisted Living and Home Care: 4th International Workshop, IWAAL 2012, Vitoria-Gasteiz, Spain, December 3-5, 2012. Proceedings 4*, pages 216–223. Springer, 2012. 4

[3] Manoj Ghuhan Arivazhagan, Vinay Aggarwal, Aaditya Kumar Singh, and Sunav Choudhary. Federated Learning with Personalization Layers. *arXiv preprint arXiv:1912.00818*, 2019. 1, 2, 4

[4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer Normalization. *arXiv preprint arXiv:1607.06450*, 2016. 3

[5] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation Learning: A Review and New Perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013. 3

[6] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating Or Propagating Gradients Through Stochastic Neurons for Conditional Computation. *arXiv preprint arXiv:1308.3432*, 2013. 2

[7] Fei Chen, Mi Luo, Zhenhua Dong, Zhenguo Li, and Xiuqiang He. Federated Meta-Learning With Fast Convergence and Efficient Communication. *arXiv preprint arXiv:1802.07876*, 2018. 2

[8] Hong-You Chen and Wei-Lun Chao. On Bridging Generic and Personalized Federated Learning for Image Classification. In *ICLR*, 2021. 1, 2, 4

[9] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A Simple Framework for Contrastive Learning of Visual Representations. In *ICML*, 2020. 3

[10] Yiqiang Chen, Wang Lu, Xin Qin, Jindong Wang, and Xing Xie. MetaFed: Federated Learning Among Federations With Cyclic Knowledge Distillation for Personalized Healthcare. *arXiv preprint arXiv:2206.08516*, 2022. 1

[11] Patryk Chrabaszcz, Ilya Loshchilov, and Frank Hutter. A Downsampled Variant of Imagenet as an Alternative to the Cifar Datasets. *arXiv preprint arXiv:1707.08819*, 2017. 4

[12] Liam Collins, Hamed Hassani, Aryan Mokhtari, and Sanjay Shakkottai. Exploiting Shared Representations for Personalized Federated Learning. In *ICML*, 2021. 1, 2, 4

[13] Alireza Fallah, Aryan Mokhtari, and Asuman Ozdaglar. Personalized Federated Learning with Theoretical Guarantees: A Model-Agnostic Meta-Learning Approach. In *NeurIPS*, 2020. 2, 4

[14] Haozhe Feng, Zhaoyang You, Minghao Chen, Tianye Zhang, Minfeng Zhu, Fei Wu, Chao Wu, and Wei Chen. KD3A: Unsupervised Multi-Source Decentralized Domain Adaptation via Knowledge Distillation. In *ICML*, 2021. 4, 5

[15] Yansong Gao, Minki Kim, Sharif Abuadbba, Yeonjae Kim, Chandra Thapa, Kyuyeon Kim, Seyit A Camtep, Hyoungshick Kim, and Surya Nepal. End-to-end evaluation of federated learning and split learning for internet of things. In *2020 International Symposium on Reliable Distributed Systems (SRDS)*, pages 91–100. IEEE, 2020. 2, 5

[16] Marta Garnelo, Dan Rosenbaum, Christopher Maddison, Tiago Ramalho, David Saxton, Murray Shanahan, Yee Whye Teh, Danilo Rezende, and SM Ali Eslami. Conditional Neural Processes. In *ICML*, 2018. 2

[17] Jonas Geiping, Hartmut Bauermeister, Hannah Dröge, and Michael Moeller. Inverting Gradients-How Easy Is It to Break Privacy in Federated Learning? In *NeurIPS*, 2020. 4

[18] Yunhui Guo, Honghui Shi, Abhishek Kumar, Kristen Grauman, Tajana Rosing, and Rogerio Feris. Spottune: Transfer Learning through Adaptive Fine-Tuning. In *CVPR*, 2019. 2, 3

[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *CVPR*, 2016. 4

[20] Alain Hore and Djemel Ziou. Image quality metrics: Psnr vs. ssim. In *ICPR*, 2010. 8

[21] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Federated Visual Classification With Real-World Data Distribution. In *ECCV*, 2020. 1

[22] Rajendra K Jain, Dah-Ming W Chiu, William R Hawe, et al. A Quantitative Measure of Fairness and Discrimination. *Eastern Research Laboratory, Digital Equipment Corporation, Hudson, MA*, 21, 1984. 7

[23] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. Bag of Tricks for Efficient Text Classification. In *EACL*, 2017. 4

[24] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and Open Problems in Federated Learning. *arXiv preprint arXiv:1912.04977*, 2019. 1

[25] Georgios A Kaissis, Marcus R Makowski, Daniel Rückert, and Rickmer F Braren. Secure, Privacy-Preserving and Federated Machine Learning in Medical Imaging. *Nature Machine Intelligence*, 2(6):305–311, 2020. 1

[26] Alex Krizhevsky and Hinton Geoffrey. Learning Multiple Layers of Features From Tiny Images. *Technical Report*, 2009. 4

[27] Qinbin Li, Yiqun Diao, Quan Chen, and Bingsheng He. Federated Learning on Non-IID Data Silos: An Experimental Study. In *ICDE*, 2022. 2, 5

[28] Qinbin Li, Bingsheng He, and Dawn Song. Model-Contrastive Federated Learning. In *CVPR*, 2021. 3, 5, 6

[29] Tian Li, Shengyuan Hu, Ahmad Beirami, and Virginia Smith. Ditto: Fair and Robust Federated Learning Through Personalization. In *ICML*, 2021. 2, 4, 7

[30] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated Optimization in Heterogeneous Networks. In *MLSys*, 2020. 2, 4

[31] Xiaoxiao Li, Meirui JIANG, Xiaofei Zhang, Michael Kamp, and Qi Dou. FedBN: Federated Learning on Non-IID Features via Local Batch Normalization. In *ICLR*, 2021. 2, 5

[32] Xin-Chun Li, De-Chuan Zhan, Yunfeng Shao, Bingshuai Li, and Shaoming Song. FedPHP: Federated Personalization with Inherited Private Models. In *ECML PKDD*, 2021. 1, 4

[33] Yuanzhi Li and Yang Yuan. Convergence Analysis of Two-Layer Neural Networks with Relu Activation. In *NeurIPS*, 2017. 3

[34] Zexi Li, Qunwei Li, Yi Zhou, Wenliang Zhong, Guannan Zhang, and Chao Wu. Edge-cloud collaborative learning with federated and centralized features. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023. 1

[35] Zexi Li, Tao Lin, Xinyi Shang, and Chao Wu. Revisiting weighted aggregation in federated learning with neural networks. In *International Conference on Machine Learning*, 2023. 1

[36] Tao Lin, Lingjing Kong, Sebastian U Stich, and Martin Jaggi. Ensemble Distillation for Robust Model Fusion in Federated Learning. In *NeurIPS*, 2020. 2, 5

[37] Lanlan Liu and Jia Deng. Dynamic Deep Neural Networks: Optimizing Accuracy-Efficiency Trade-Offs by Selective Execution. In *AAAI*, 2018. 2

[38] Yang Liu, Anbu Huang, Yun Luo, He Huang, Youzhi Liu, Yuanyuan Chen, Lican Feng, Tianjian Chen, Han Yu, and Qiang Yang. Fedvision: An Online Visual Object Detection Platform Powered By Federated Learning. In *AAAI*, 2020. 1

[39] Mi Luo, Fei Chen, Dapeng Hu, Yifan Zhang, Jian Liang, and Jiashi Feng. No Fear of Heterogeneity: Classifier Calibration for Federated Learning with Non-IID data. In *NeurIPS*, 2021. 4

[40] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *AISTATS*, 2017. 2, 4, 5

[41] Milad Nasr, Reza Shokri, and Amir Houmansadr. Comprehensive privacy analysis of deep learning. In *Proceedings of the 2019 IEEE Symposium on Security and Privacy (SP)*, pages 1–15, 2018. 4

[42] Yifan Niu and Weihong Deng. Federated Learning for Face Recognition With Gradient Correction. In *AAAI*, 2022. 8

[43] Boris Oreshkin, Pau Rodríguez López, and Alexandre Lacoste. Tadam: Task Dependent Adaptive Metric for Improved Few-Shot Learning. In *NeurIPS*, 2018. 2, 3

[44] Xingchao Peng, Qinxun Bai, Xide Xia, Zijun Huang, Kate Saenko, and Bo Wang. Moment Matching for Multi-Source Domain Adaptation. In *ICCV*, 2019. 4

[45] Krishna Pillutla, Kshitiz Malik, Abdel-Rahman Mohamed, Mike Rabbat, Maziar Sanjabi, and Lin Xiao. Federated Learning with Partial Model Personalization. In *ICML*, 2022. 7

[46] Jiawei Ren, Cunjun Yu, Xiao Ma, Haiyu Zhao, Shuai Yi, et al. Balanced Meta-Softmax for Long-Tailed Visual Recognition. In *NeurIPS*, 2020. 2

[47] Aviv Shamsian, Aviv Navon, Ethan Fetaya, and Gal Chechik. Personalized Federated Learning using Hypernetworks. In *ICML*, 2021. 5

[48] Kihyuk Sohn. Improved Deep Metric Learning With Multi-Class N-Pair Loss Objective. In *NeurIPS*, 2016. 3

[49] Canh T Dinh, Nguyen Tran, and Tuan Dung Nguyen. Personalized Federated Learning with Moreau Envelopes. In *NeurIPS*, 2020. 1, 2, 4

[50] Alysa Ziying Tan, Han Yu, Lizhen Cui, and Qiang Yang. Towards Personalized Federated Learning. *IEEE Transactions on Neural Networks and Learning Systems*, 2022. 1

[51] Yue Tan, Guodong Long, Lu Liu, Tianyi Zhou, Qinghua Lu, Jing Jiang, and Chengqi Zhang. Fedproto: Federated Prototype Learning across Heterogeneous Clients. In *AAAI*, 2022. 1, 3, 4

[52] Laurens Van der Maaten and Geoffrey Hinton. Visualizing Data Using T-SNE. *Journal of Machine Learning Research*, 9(11), 2008. 5

[53] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All You Need. In *NeurIPS*, 2017. 3

[54] Chi Wang, Yang Hua, Zheng Lu, Jian Gao, and Neil Robertson. Temporal Meta-Adaptor for Video Object Detection. In *BMVC*, 2021. 3

[55] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H. Vincent Poor. Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization. In *NeurIPS*, 2020. 5

[56] Yuezhou Wu, Yan Kang, Jiahuan Luo, Yuanqin He, Lixin Fan, Rong Pan, and Qiang Yang. Fedcg: Leverage conditional gan for protecting privacy and maintaining competitive performance in federated learning. In *NeurIPS*, 2022. 4

[57] Zuxuan Wu, Tushar Nagarajan, Abhishek Kumar, Steven Rennie, Larry S Davis, Kristen Grauman, and Rogerio Feris. Blockdrop: Dynamic Inference Paths in Residual Networks. In *CVPR*, 2018. 2

[58] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017. 4

[59] Donggeun Yoo and In So Kweon. Learning loss for active learning. In *CVPR*, 2019. 4

[60] Ming Zeng, Le T Nguyen, Bo Yu, Ole J Mengshoel, Jiang Zhu, Pang Wu, and Joy Zhang. Convolutional neural networks for human activity recognition using mobile sensors. In *6th international conference on mobile computing, applications and services*, pages 197–205. IEEE, 2014. 5

[61] Jie Zhang, Song Guo, Xiaosong Ma, Haozhao Wang, Wenchao Xu, and Feijie Wu. Parameterized Knowledge Transfer for Personalized Federated Learning. In *NeurIPS*, 2021. 3

[62] Jianqing Zhang, Yang Hua, Hao Wang, Tao Song, Zhengui Xue, Ruhui Ma, and Haibing Guan. FedALA: Adaptive Local Aggregation for Personalized Federated Learning. In *AAAI*, 2023. 1

[63] Jianqing Zhang, Yang Hua, Hao Wang, Tao Song, Zhengui Xue, Ruhui Ma, and Haibing Guan. Fedcp: Separating feature information for personalized federated learning via conditional policy. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023. 1

[64] Jianqing Zhang, Dongjing Wang, and Dongjin Yu. TLSAN: Time-aware Long-and Short-term Attention Network for Next-item Recommendation. *Neurocomputing*, 441:179–191, 2021. 3

[65] Qiming Zhang, Jing Zhang, Wei Liu, and Dacheng Tao. Category anchor-guided unsupervised domain adaptation for semantic segmentation. In *NeurIPS*, 2019. 1

[66] Tuo Zhang, Lei Gao, Chaoyang He, Mi Zhang, Bhaskar Krishnamachari, and A Salman Avestimehr. Federated learning for the internet of things: applications, challenges, and opportunities. *IEEE Internet of Things Magazine*, 5(1):24–29, 2022. 2, 5

[67] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-Level Convolutional Networks for Text Classification. In *NeurIPS*, 2015. 4

[68] Zhilu Zhang and Mert Sabuncu. Generalized Cross Entropy Loss for Training Deep Neural Networks With Noisy Labels. In *NeurIPS*, 2018. 4

[69] Ligeng Zhu, Zhijian Liu, and Song Han. Deep Leakage from Gradients. In *NeurIPS*, 2019. 4